Java Developer Interview Knowledge Base

AI Interview Coach ��� RAG Knowledge Base

2026-07-11

Contents

Java Developer Interview Knowledge Base

A structured, retrieval-friendly reference for an AI Interview Coach. Each section is self-contained with a definition, explanation, code example, best practices, common mistakes, and interview tips so it can be chunked independently for semantic search.

Introduction to Java

Definition: Java is a high-level, class-based, object-oriented, platform-independent programming language. Code is compiled into bytecode

(`.class` files) that runs on the Java Virtual Machine (JVM), which is why Java follows the principle **"Write Once, Run Anywhere" (WORA)**.

Key characteristics: - **Platform independent** – bytecode runs on any device with a compatible JVM. - **Object-oriented** – everything (except primitives) is modeled as objects. - **Statically typed** – variable types are checked at compile time. - **Automatic memory management** – garbage collection removes unused objects. - **Multithreaded** – built-in support for concurrent programming. - **Robust and secure** – strong exception handling, no explicit pointers, bytecode verification.

Example:

```java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

Interview Tip: Interviewers often ask, *"Why is Java platform independent?"* — the correct answer centers on bytecode + JVM, not the source code itself. Source code is compiled to bytecode once; each platform has its own JVM implementation that interprets/executes that same bytecode.

Common Mistake: Confusing "platform independent" with "JVM independent." The JVM itself is platform-*dependent* (a different JVM binary exists for Windows, Linux, macOS); it is the *bytecode* that is platform-independent.

JVM, JDK, and JRE

JVM (Java Virtual Machine): An abstract machine that provides a runtime environment to execute Java bytecode. It performs class loading, bytecode verification, execution (via interpreter/JIT compiler), and memory management (garbage collection). JVM is platform-dependent — each OS has its

own JVM implementation.

JRE (Java Runtime Environment): JVM + core libraries (`java.lang`, `java.util`, etc.) + supporting files needed to *run* Java applications. JRE does **not** include development tools like the compiler.

JDK (Java Development Kit): JRE + development tools such as `javac` (compiler), `javadoc`, `jdb` (debugger), and `jar`. JDK is required to *write and compile* Java programs.

Relationship: `JDK ⊃ JRE ⊃ JVM`

Component	Purpose	Contains
JVM	Executes bytecode	Class loader, execution engine, runtime memory areas
JRE	Runs Java apps	JVM + libraries
JDK	Develops Java apps	JRE + compiler + dev tools

JVM Architecture (high level): 1. **Class Loader Subsystem** – loads `.class` files into memory (Bootstrap, Extension, Application class loaders). 2. **Runtime Data Areas** – Method Area, Heap, Stack, PC Registers, Native Method Stack. 3. **Execution Engine** – Interpreter, JIT (Just-In-Time) Compiler, Garbage Collector.

Interview Tip: A classic question is *"Can you run a Java program with only JRE installed?"* Yes, if it's already compiled (`.class`/`.jar`), because JRE contains the JVM needed to execute it — but you cannot **compile** `.java` source files without the JDK.

Common Mistake: Saying JDK and JRE are "the same thing" — always be ready to explain the containment relationship precisely.

Variables and Data Types

Definition: A variable is a named memory location that holds a value of a specific data type.

Types of Variables

- **Local variable** – declared inside a method; must be initialized before use; exists only during method execution.
- **Instance variable** – declared inside a class but outside methods; each object has its own copy; default values apply.
- **Static variable** – declared with `static`; shared across all instances of a class.

Data Types

Primitive types (8 total) — stored directly in the stack, hold actual values:

Type	Size	Default	Example
`byte`	1 byte	0	`byte b = 10;`
`short`	2 bytes	0	`short s = 200;`
`int`	4 bytes	0	`int i = 1000;`
`long`	8 bytes	0L	`long l = 100000L;`
`float`	4 bytes	0.0f	`float f = 3.14f;`
`double`	8 bytes	0.0d	`double d = 3.14159;`
`char`	2 bytes	‘Ŏ000’	`char c = 'A';`
`boolean`	1 bit (JVM-dependent)	false	`boolean flag = true;`

Reference types — store the *address* of an object in the heap: classes, interfaces, arrays, `String`, wrapper classes (`Integer`, `Double`, etc.).

Example:

```java
public class VariableDemo {
    static int staticVar = 100;      // static variable
    int instanceVar = 50;            // instance variable

    void show() {
        int localVar = 10;           // local variable, must be initialize
        System.out.println(localVar + instanceVar + staticVar);
    }
}
```

Best Practice: Use the smallest data type that safely fits your value range to save memory (e.g., prefer `int` over `long` unless large values are expected). Use `final` for constants: `final double PI = 3.14159;`.

Common Mistake: Forgetting that local variables have **no default value** — using an uninitialized local variable causes a compile-time error, unlike instance/static variables which get defaults automatically.

Interview Tip: Know autoboxing/unboxing: `Integer x = 10;` (autobox) and `int y = x;` (unbox). Be ready to explain == vs `.equals()` for wrapper classes (Integer caching for values -128 to 127).

Operators

Definition: Operators are symbols that perform operations on variables and values.

Categories

- **Arithmetic:** `+ - * / %`
- **Relational:** `== != > < >= <=`
- **Logical:** `&& || !`
- **Bitwise:** `& | ^ ~ << >> >>>`

- **Assignment:** `= += -= *= /= %=`
- **Unary:** `+ - ++ --`
- **Ternary:** `condition ? expr1 : expr2`
- **instanceof:** checks object type

Example:

```java
int a = 10, b = 3;
System.out.println(a % b);       // 1 (modulus)
System.out.println(a / b);       // 3 (integer division)
System.out.println(a > b ? "A wins" : "B wins"); // ternary
System.out.println(5 & 3);       // 1 (bitwise AND)
System.out.println(-8 >>> 1);    // unsigned right shift
```

Best Practice: Use >>> (unsigned right shift) only when you explicitly need zero-fill behavior on negative numbers; prefer >> for normal signed shifting.

Common Mistake: Confusing & / | (bitwise, always evaluate both operands) with && / || (logical, short-circuit — right operand skipped if the result is already determined). This matters when the right operand has side effects, e.g., `if (obj != null & obj.getValue() > 0)` can throw `NullPointerException`, while && would short-circuit safely.

Interview Tip: Be ready to trace operator precedence and short-circuit evaluation with a code snippet — a very common whiteboard question.

Control Statements

Definition: Control statements direct the flow of program execution based on conditions or repetition.

Conditional Statements

```java
if (score >= 90) {
    grade = "A";
} else if (score >= 75) {
    grade = "B";
} else {
    grade = "C";
}

switch (day) {
    case 1 -> System.out.println("Monday");
    case 2 -> System.out.println("Tuesday");
    default -> System.out.println("Other day");
}
```

Looping Statements

```java
for (int i = 0; i < 5; i++) { System.out.println(i); }

int i = 0;
while (i < 5) { System.out.println(i); i++; }

int j = 0;
do { System.out.println(j); j++; } while (j < 5);

for (int num : new int[]{1, 2, 3}) { System.out.println(num); } // enhance
```

Jump Statements

- `break` – exits the nearest loop or switch.
- `continue` – skips the current iteration.

- `return` – exits a method, optionally returning a value.
- Labeled `break`/`continue` – control outer loops from nested loops.

Example (labeled break):

```
outer:
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (j == 1) continue outer;
        System.out.println(i + "-" + j);
    }
}
```

Best Practice: Prefer the modern switch expression (`->` syntax, Java 14+) to avoid fall-through bugs and to allow `switch` to return a value directly.

Common Mistake: Forgetting `break` in traditional `switch` statements, causing unintended fall-through execution into subsequent cases.

Interview Tip: Be ready to explain the difference between `while` and `do-while` (do-while always executes at least once) with a real example.

Arrays

Definition: An array is a fixed-size, ordered collection of elements of the same type, stored in contiguous memory and indexed from `0`.

Example:

```
int[] numbers = {10, 20, 30, 40};
System.out.println(numbers[0]);       // 10
System.out.println(numbers.length);   // 4 (property, not method)

// 2D array
int[][] matrix = {{1, 2}, {3, 4}};
```

```java
System.out.println(matrix[1][0]);     // 3

// Declaration + allocation
String[] names = new String[3];       // default values: null
names[0] = "Alice";
```

Iterating:

```java
for (int n : numbers) {
    System.out.println(n);
}
```

Useful utility methods (`java.util.Arrays`):

```java
Arrays.sort(numbers);
int idx = Arrays.binarySearch(numbers, 20);
int[] copy = Arrays.copyOf(numbers, 5);
System.out.println(Arrays.toString(numbers));
```

Best Practice: Use `Arrays.toString()`/`Arrays.deepToString()` for debugging instead of printing the array reference directly (which prints a hashcode-like string).

Common Mistake: `ArrayIndexOutOfBoundsException` from off-by-one errors, and confusing array `.length` (field) with `String.length()` (method) or `List.size()` (method).

Interview Tip: Know that arrays in Java are objects stored on the heap, have fixed size once created, and default values depend on type (`0` for numeric, `false` for boolean, `null` for objects).

Strings

Definition: `String` is an immutable sequence of characters in Java, represented by the `java.lang.String` class. Immutable means once created, a `String` object's content cannot be changed — any modification creates a new object.

Example:

```
String s1 = "Hello";               // stored in String Pool
String s2 = new String("Hello");  // stored in Heap (separate object)
System.out.println(s1 == s2);          // false (different references)
System.out.println(s1.equals(s2));    // true (same content)

String s3 = s1.concat(" World");  // creates a NEW string, s1 unchanged
System.out.println(s1);           // "Hello"
System.out.println(s3);           // "Hello World"
```

String Pool: String literals are stored in a special memory area called the String Constant Pool (part of heap since Java 7) to allow reuse. `new String()` bypasses the pool and always creates a new object; `.intern()` can force pooling.

StringBuilder / StringBuffer: Use for mutable string operations — much more efficient than repeated `String` concatenation in loops.

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 5; i++) {
    sb.append(i).append(",");
}
System.out.println(sb.toString()); // 0,1,2,3,4,
```

- `StringBuilder` – not synchronized, faster, use in single-threaded contexts.
- `StringBuffer` – synchronized, thread-safe, slightly slower.

Common String methods: `length()`, `charAt()`, `substring()`, `indexOf()`, `split()`, `trim()`, `toUpperCase()`, `replace()`, `equalsIgnoreCase()`.

Best Practice: Use `.equals()` (or `.equalsIgnoreCase()`) for content comparison, never == (which compares references). Use `StringBuilder` inside loops instead of + concatenation to avoid creating many intermediate objects.

Common Mistake: Using == to compare Strings and getting inconsistent results depending on whether literals or `new String()` were used. Also, assuming `String` methods modify the original string in place (they don't — `String` is immutable).

Interview Tip: Be ready to explain *why* String is immutable: security (used in class loading, network connections), thread-safety (safely shared without synchronization), caching of hashcode, and safe use as a `HashMap` key.

Object-Oriented Programming (OOP)

Java is built around four core OOP pillars: **Encapsulation, Inheritance, Polymorphism, Abstraction**, implemented via **Classes** and **Objects**.

Class

Definition: A class is a blueprint/template that defines the structure (fields) and behavior (methods) of objects.

```
public class Car {
    String model;
    int speed;

    void accelerate() {
        speed += 10;
```

```
    }
}
```

Object

Definition: An object is a runtime instance of a class, created using the new keyword, occupying memory on the heap.

```
Car myCar = new Car();   // object creation
myCar.model = "Tesla";
myCar.accelerate();
```

Interview Tip: A class is a logical concept (no memory until instantiated); an object is a physical entity with actual state in memory. Interviewers often ask you to explain this distinction with an analogy (e.g., blueprint vs. house).

Encapsulation

Definition: Wrapping data (fields) and methods that operate on it into a single unit (class), while restricting direct access to internal state using access modifiers (typically `private` fields with public getters/setters).

```
public class Account {
    private double balance;   // hidden from outside

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        if (amount > 0) balance += amount;   // controlled modification
```

```
    }
}
```

Best Practice: Keep fields `private` and expose controlled access via getters/setters, validating input inside setters to protect object invariants.

Common Mistake: Making fields `public` "for convenience," which defeats encapsulation and allows invalid states (e.g., negative balance) to be set directly from outside the class.

Interview Tip: Emphasize that encapsulation enables data hiding, easier maintenance, and validation logic centralization.

Inheritance

Definition: A mechanism where one class (subclass/child) acquires the fields and methods of another class (superclass/parent) using the `extends` keyword, enabling code reuse.

```
class Animal {
    void eat() { System.out.println("This animal eats food"); }
}

class Dog extends Animal {
    void bark() { System.out.println("Dog barks"); }
}

Dog d = new Dog();
d.eat();   // inherited from Animal
d.bark();  // defined in Dog
```

Java supports **single inheritance** for classes (one direct superclass) but **multiple inheritance via interfaces**.

super keyword: Refers to the immediate parent class — used to call parent constructors (`super(...)`) or overridden methods (`super.method()`).

Best Practice: Favor **composition over inheritance** when there's no true "is-a" relationship — inheritance creates tight coupling between classes.

Common Mistake: Overusing inheritance for code reuse alone (leading to fragile hierarchies) instead of considering interfaces or composition.

Interview Tip: Be ready to explain why Java doesn't support multiple class inheritance (the "diamond problem" — ambiguity when two parent classes define the same method) and how interfaces solve it via explicit default method resolution rules.

Polymorphism

Definition: The ability of an object to take many forms — the same method call behaves differently based on the object/context. Java supports two types:

1. **Compile-time (static) polymorphism** – method overloading.
2. **Runtime (dynamic) polymorphism** – method overriding, resolved via dynamic method dispatch.

```
class Shape {
    double area() { return 0; }
}

class Circle extends Shape {
    double radius;
    Circle(double r) { radius = r; }
    @Override
    double area() { return Math.PI * radius * radius; }
```

```
}

class Square extends Shape {
    double side;
    Square(double s) { side = s; }
    @Override
    double area() { return side * side; }
}

Shape[] shapes = { new Circle(3), new Square(4) };
for (Shape s : shapes) {
    System.out.println(s.area());   // calls the correct overridden method
}
```

Best Practice: Program to an interface/superclass type (`Shape s = new Circle(3);`) to leverage polymorphism and write flexible, extensible code.

Common Mistake: Confusing overloading (compile-time, same class, different parameter lists) with overriding (runtime, subclass, same signature) — a very frequent interview trap.

Interview Tip: Explain **dynamic method dispatch**: at compile time, the reference type determines which methods are *accessible*; at runtime, the JVM determines which overridden method *implementation* actually executes, based on the object's actual type.

Abstraction

Definition: Hiding implementation details and exposing only essential features. In Java, achieved using **abstract classes** and **interfaces**.

```
abstract class Payment {
    abstract void pay(double amount);   // no implementation
```

```
    void printReceipt() {                // concrete shared method
        System.out.println("Payment processed.");
    }
}

class CreditCardPayment extends Payment {
    @Override
    void pay(double amount) {
        System.out.println("Paid " + amount + " via credit card");
    }
}
```

Best Practice: Use abstraction to define a stable contract for consumers while allowing implementation details to change freely behind the scenes.

Interview Tip: Interviewers often ask “abstraction vs. encapsulation” — abstraction hides *complexity/implementation* (design-level, “what” an object does), while encapsulation hides *data* (implementation-level, “how” data is protected).

Constructors

Definition: A constructor is a special block of code, with the same name as the class and no return type, used to initialize objects when they are created.

Types: - **Default constructor** – no-argument constructor automatically provided by the compiler if no constructor is defined. - **Parameterized constructor** – accepts arguments to initialize fields with specific values. - **Copy constructor** (not built-in like C++, but commonly implemented manually) – creates a new object by copying an existing one.

```java
public class Employee {
    String name;
    double salary;

    Employee() {                          // default constructor
        this("Unknown", 0.0);
    }

    Employee(String name, double salary) {  // parameterized constructor
        this.name = name;
        this.salary = salary;
    }

    Employee(Employee other) {          // copy constructor
        this.name = other.name;
        this.salary = other.salary;
    }
}
```

Constructor chaining: Using `this(...)` to call another constructor in the same class, or `super(...)` to call the parent class constructor — must be the **first statement** in the constructor.

Best Practice: Use constructor chaining (`this(...)`) to avoid duplicating initialization logic across overloaded constructors.

Common Mistake: Believing constructors are inherited — they are **not**; a subclass must define its own constructors, though it can call `super(...)` to reuse parent initialization logic. Also, once you define *any* constructor explicitly, Java no longer provides the default no-arg constructor automatically.

Interview Tip: Be ready to explain the difference between a constructor and a method: constructors have no return type (not even `void`), share the

class name, and are invoked only via new.

Method Overloading

Definition: Defining multiple methods in the same class with the same name but different parameter lists (different number, type, or order of parameters). Resolved at **compile time** — a form of static polymorphism.

```
class Calculator {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
}
```

Rules: - Return type alone is **not** sufficient to overload a method — parameter lists must differ. - Overloading can happen within the same class or between a superclass and subclass.

Best Practice: Use overloading to provide convenient variations of an operation (e.g., `println(int)`, `println(String)`) rather than creating verbosely-named separate methods.

Common Mistake: Attempting to overload by changing only the return type — this causes a compile-time error since the compiler cannot resolve calls based on return type alone.

Interview Tip: Know how overload resolution works with type widening, autoboxing, and varargs (in that priority order) — e.g., calling `add(5, 5)` with overloads `add(int,int)` and `add(long,long)` picks the `int` version first without widening.

Method Overriding

Definition: A subclass provides a specific implementation of a method already defined in its superclass, with the **same signature** (name, parameters, and compatible return type). Resolved at **runtime** — dynamic polymorphism.

```
class Animal {
    void sound() { System.out.println("Some generic sound"); }
}

class Cat extends Animal {
    @Override
    void sound() { System.out.println("Meow"); }
}
```

Rules for valid overriding: - Method name and parameter list must be identical. - Return type must be the same or a covariant type (subtype). - Access modifier cannot be more restrictive than the overridden method. - Cannot override `static`, `final`, or `private` methods (static methods are *hidden*, not overridden). - Checked exceptions in the overriding method cannot be broader than those declared in the parent method.

Best Practice: Always use the `@Override` annotation — it lets the compiler catch mistakes (e.g., typos in method signature) at compile time instead of silently creating a new overloaded method.

Common Mistake: Accidentally overloading instead of overriding due to a mismatched signature (e.g., different parameter type), then wondering why the "overridden" behavior never executes.

Interview Tip: Be ready to explain **why constructors, static, and private methods cannot be overridden**: overriding relies on dynamic dispatch via the object's runtime type, but constructors aren't inherited, static methods are resolved via the class reference (compile-time), and private

methods aren't visible outside their own class.

Access Modifiers

Definition: Keywords that set the visibility/accessibility of classes, methods, and fields.

Modifier	Same Class	Same Package	Subclass (different package)	Different Package
`private`	Yes	No	No	No
default (no modifier)	Yes	Yes	No	No
`protected`	Yes	Yes	Yes	No
`public`	Yes	Yes	Yes	Yes

```java
public class Example {
    private int secret;         // only within this class
    int packageLevel;           // default, only within same package
    protected int forSubclass;  // package + subclasses
    public int openField;       // accessible everywhere
}
```

Best Practice: Follow the principle of least privilege — default to `private` for fields, and only widen visibility (`protected`/`public`) when there's a real need for external access.

Common Mistake: Overusing `public` fields, which breaks encapsulation and makes future refactoring riskier since external code may depend directly on internal state.

Interview Tip: Note that top-level classes can only be `public` or default (package-private) — not `private` or `protected`.

Packages

Definition: A package is a namespace that organizes related classes and interfaces, helping avoid naming conflicts and controlling access.

```
package com.company.project;

import java.util.List;          // single-class import
import java.util.*;             // wildcard import (all classes in java.uti

public class OrderService {
    List<String> orders = new ArrayList<>();
}
```

Types: - **Built-in packages** – `java.lang` (auto-imported), `java.util`, `java.io`, `java.net`, etc. - **User-defined packages** – organized by reverse domain convention, e.g., `com.company.module`.

Best Practice: Structure packages by feature/domain (`com.company.order`, `com.company.payment`) rather than by layer only, for better cohesion in larger applications. Avoid wildcard imports in production code for clarity on exact dependencies.

Common Mistake: Placing the package statement anywhere other than the very first line of the file (excluding comments), which causes a compile-time error.

Interview Tip: Know that classes in the same package can access each other's default (package-private) members without explicit imports.

Interfaces

Definition: An interface defines a contract of abstract methods (and optionally `default`, `static`, and `private` methods since Java 8/9) that implementing classes must fulfill. All fields in an interface are implicitly `public static final`.

```
interface Drivable {
    void drive();                          // abstract method

    default void honk() {                  // default method (Java 8+)
        System.out.println("Beep beep!");
    }

    static void info() {                   // static method
        System.out.println("Drivable interface");
    }
}

class Car implements Drivable {
    @Override
    public void drive() {
        System.out.println("Car is driving");
    }
}
```

Key points: - A class can implement **multiple interfaces**, enabling multiple inheritance of type. - If two interfaces have conflicting default methods, the implementing class **must override** the method to resolve ambiguity. - Interface methods are `public` and `abstract` by default (unless `default`/`static`/`private`).

Best Practice: Design interfaces around behavior/capability ("can-do" relationships, e.g., `Runnable`, `Comparable`) rather than around data.

Common Mistake: Forgetting to mark implementing methods `public` — interface methods are implicitly `public`, so a subclass cannot reduce visibility when implementing them.

Interview Tip: Be ready to explain the purpose of `default` methods: they allow interfaces to evolve (add new methods) without breaking existing implementing classes — this was the core motivation behind their introduction in Java 8 (e.g., adding `forEach` to `Collection`).

Abstract Classes

Definition: A class declared with the `abstract` keyword that cannot be instantiated directly and may contain both abstract methods (no body) and concrete methods (with implementation).

```
abstract class Vehicle {
    abstract void move();                // must be implemented by subclass

    void fuelUp() {                      // concrete shared method
        System.out.println("Refueling...");
    }
}

class Bike extends Vehicle {
    @Override
    void move() { System.out.println("Bike is moving"); }
}
```

Abstract Class vs. Interface

Aspect	Abstract Class	Interface
Instantiation	Cannot be instantiated	Cannot be instantiated
Methods	Abstract + concrete	Abstract, default, static, private
Fields	Any type (instance state allowed)	Only `public static final` constants
Inheritance	Single (`extends`)	Multiple (`implements`)
Constructors	Allowed	Not allowed
Use case	Shared base state/behavior, "is-a"	Capability contract, "can-do"

Best Practice: Use an abstract class when subclasses share common state and some common implementation; use an interface when you only need to define a capability contract, potentially across unrelated class hierarchies.

Common Mistake: Trying to instantiate an abstract class directly (`new Vehicle()`) — this is a compile-time error.

Interview Tip: A common follow-up question: *"When would you use an abstract class over an interface (post Java 8, since interfaces now support default methods)?"* — Answer: when you need constructors, instance fields with real state, or a non-`public` access level for some methods.

Exception Handling

Definition: A mechanism to handle runtime errors, allowing the program to continue execution or fail gracefully rather than crashing abruptly.

Exception Hierarchy

`Throwable`

```
├── Error (serious, unrecoverable – e.g., OutOfMemoryError, StackOverflowEr
└── Exception
        ├── Checked Exceptions (must be handled/declared –
e.g., IOException, SQLException)
    └── Unchecked Exceptions / RuntimeException (e.g., NullPointerException,
```

Example:

```java
public class ExceptionDemo {
    public static void main(String[] args) {
        try {
            int result = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero: " + e.getMessage())
        } finally {
            System.out.println("This always executes.");
        }
    }
}
```

Custom exceptions:

```java
class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String message) {
        super(message);
    }
}

void withdraw(double amount, double balance) throws InsufficientFundsExcep
    if (amount > balance) {
        throw new InsufficientFundsException("Not enough balance");
    }
}
```

Try-with-resources (Java 7+): Automatically closes resources imple-

menting AutoCloseable.

```
try (BufferedReader reader = new BufferedReader(new FileReader("data.txt")
    System.out.println(reader.readLine());
} catch (IOException e) {
    e.printStackTrace();
}
```

Checked vs. Unchecked: - **Checked** – subclasses of Exception (excluding RuntimeException); must be caught or declared with throws. Represent recoverable conditions (e.g., file not found). - **Unchecked** – subclasses of RuntimeException; not required to be declared/caught. Usually represent programming bugs (e.g., null dereference, invalid array index).

Best Practice: Catch specific exception types rather than generic Exception/Throwable; always clean up resources (try-with-resources); never swallow exceptions silently (empty catch blocks).

Common Mistake: Catching Exception broadly and ignoring the actual error, or using exceptions for normal control flow (which is expensive and unclear).

Interview Tip: Be ready to explain execution order with finally — it runs even if a return statement exists in try/catch, unless System.exit() is called or the JVM crashes.

Collections Framework

Definition: A unified architecture (java.util package) of interfaces and classes for storing, retrieving, and manipulating groups of objects. Core interfaces: Collection, List, Set, Queue, Map (Map is technically separate from Collection).

Hierarchy overview:

```
Collection
 ├── List (ordered, allows duplicates) — ArrayList, LinkedList, Vector
 ├── Set (no duplicates) — HashSet, LinkedHashSet, TreeSet
 └── Queue (FIFO/priority processing) — LinkedList, PriorityQueue

Map (key-value pairs, not a Collection) — HashMap, LinkedHashMap, TreeMap
```

Best Practice: Always code against the interface type, not the implementation: `List<String> list = new ArrayList<>();` instead of `ArrayList<String> list = new ArrayList<>();` — this allows swapping implementations without changing calling code.

Common Mistake: Using `Vector`/`Hashtable` (legacy, synchronized, slower) in new code instead of modern alternatives (`ArrayList`, `HashMap`, or `Collections.synchronizedList`/`ConcurrentHashMap` for thread safety).

Interview Tip: Be ready to compare `Collection` vs `Collections` — `Collection` is the root interface, `Collections` (plural) is a utility class with static helper methods (`sort`, `reverse`, `unmodifiableList`, etc.).

ArrayList

Definition: A resizable array implementation of the `List` interface, backed internally by a dynamic array. Maintains insertion order and allows duplicates and `null`.

```java
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add(1, "C");     // insert at index
System.out.println(list.get(0));   // "A"
list.remove("B");
```

```
System.out.println(list);           // [A, C]
```

Performance: - `get(index)` – O(1) (direct array access). - `add(element)` at end – amortized O(1) (occasional resize is O(n)). - add/remove at arbitrary index – O(n) (requires shifting elements).

Best Practice: Specify an initial capacity (`new ArrayList<>(expectedSize)`) if the approximate size is known in advance, to reduce internal resizing overhead.

Common Mistake: Modifying a list while iterating with a standard `for-each` loop, which throws `ConcurrentModificationException`. Use an `Iterator`'s `remove()` method or `removeIf()` instead.

```
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    if (it.next().equals("C")) it.remove();  // safe removal
}
```

Interview Tip: Know when to prefer `ArrayList` over `LinkedList`: `ArrayList` is better for frequent random access (get/set); `LinkedList` is better for frequent insertions/deletions at the beginning/middle.

LinkedList

Definition: A doubly-linked list implementation of both the `List` and `Deque` interfaces — each node holds references to the previous and next node.

```
LinkedList<String> ll = new LinkedList<>();
ll.addFirst("A");
ll.addLast("B");
ll.add(1, "C");
System.out.println(ll);         // [A, C, B]
ll.removeFirst();
```

Performance: - add/remove at the beginning or end – O(1). - get(index) – O(n) (must traverse from head or tail).

Use as Queue/Deque:

```
Deque<Integer> stack = new LinkedList<>();
stack.push(1);   // acts as a stack (LIFO)
stack.push(2);
System.out.println(stack.pop());  // 2

Queue<Integer> queue = new LinkedList<>();
queue.offer(1);  // acts as a queue (FIFO)
queue.offer(2);
System.out.println(queue.poll()); // 1
```

Best Practice: Use LinkedList when the access pattern is mostly sequential (iteration) or when frequently adding/removing at the ends (as a Deque/Queue), not for random-access-heavy workloads.

Common Mistake: Using LinkedList by default "because it sounds more efficient" — in practice, ArrayList has better cache locality and often outperforms LinkedList even for many insert/delete patterns due to memory overhead of node pointers.

Interview Tip: Be ready to state Big-O trade-offs side-by-side for ArrayList vs LinkedList — a near-guaranteed question.

HashMap

Definition: A hash-table-based implementation of the Map interface storing key-value pairs. No guaranteed order of iteration, allows one null key and multiple null values, not thread-safe.

```java
Map<String, Integer> ages = new HashMap<>();
ages.put("Alice", 30);
ages.put("Bob", 25);
ages.put("Alice", 31);          // overwrites existing key
System.out.println(ages.get("Alice"));   // 31
System.out.println(ages.getOrDefault("Eve", 0));  // 0

for (Map.Entry<String, Integer> entry : ages.entrySet()) {
    System.out.println(entry.getKey() + " = " + entry.getValue());
}
```

How it works internally: Keys are hashed via `hashCode()` to determine a bucket index; on collisions, entries in the same bucket form a linked list (converted to a balanced tree if a bucket grows beyond a threshold, since Java 8, for better worst-case performance). `equals()` is used to confirm key matches within a bucket.

Performance: Average O(1) for get/put/remove; worst case O(log n) with treeified buckets (Java 8+) or O(n) pre-Java 8.

Best Practice: Always override both `hashCode()` and `equals()` consistently when using custom objects as HashMap keys — violating this contract causes lookups to silently fail.

Common Mistake: Using a mutable object as a key and then changing its state after insertion — this can make the entry unreachable because its hash bucket no longer matches its current hash code.

Interview Tip: Be ready to explain `HashMap` vs `Hashtable` (legacy, synchronized, no null keys/values) and `HashMap` vs `ConcurrentHashMap` (thread-safe, segment/bucket-level locking, no locking of the entire map).

HashSet

Definition: A `Set` implementation backed internally by a `HashMap` (elements stored as keys with a dummy constant value), guaranteeing no duplicate elements and no guaranteed order.

```
Set<String> names = new HashSet<>();
names.add("Alice");
names.add("Bob");
names.add("Alice");     // duplicate, ignored
System.out.println(names.size());  // 2
System.out.println(names.contains("Bob"));  // true
```

Related Set implementations: - `LinkedHashSet` – maintains insertion order. - `TreeSet` – sorted order (natural ordering or via `Comparator`), backed by a `TreeMap` (red-black tree).

Best Practice: Use `HashSet` for fast membership checks (`contains`) when order doesn't matter; use `LinkedHashSet` when insertion order must be preserved; use `TreeSet` when sorted iteration is required.

Common Mistake: Assuming `HashSet` preserves insertion order — it does not; relying on iteration order with `HashSet` is a bug waiting to happen.

Interview Tip: Know that `HashSet.add()` relies on `hashCode()`/`equals()` just like `HashMap` keys — for custom objects, both must be properly overridden or duplicates may not be detected correctly.

Generics

Definition: Generics allow classes, interfaces, and methods to operate on typed parameters, enabling compile-time type safety and eliminating the need for explicit casting.

```java
class Box<T> {
    private T content;
    public void set(T content) { this.content = content; }
    public T get() { return content; }
}

Box<String> stringBox = new Box<>();
stringBox.set("Hello");
String value = stringBox.get();   // no cast needed

// Generic method
public static <T> void printArray(T[] array) {
    for (T item : array) System.out.println(item);
}
```

Bounded type parameters:

```java
class NumericBox<T extends Number> {
    T value;
    double doubleValue() { return value.doubleValue(); }
}
```

Wildcards:

```java
void printList(List<?> list) { ... }                    // unbounded
void processNumbers(List<? extends Number> list) { }  // upper-bounded (re
void addIntegers(List<? super Integer> list) { }       // lower-bounded (w
```

Best Practice: Use `? extends T` when you only need to *read* from a generic structure (producer), and `? super T` when you only need to *write* to it (consumer) — remembered via the mnemonic **PECS** (Producer Extends, Consumer Super).

Common Mistake: Assuming generics work with primitives directly (`Box<int>` is invalid — use wrapper types like `Box<Integer>`). Also,

forgetting that Java generics use **type erasure**: generic type information is removed at runtime, so `new T[]` and `instanceof List<String>` (as opposed to raw `List`) are not directly possible.

Interview Tip: Be ready to explain **type erasure** — the compiler enforces type checks at compile time and then strips generic type parameters, replacing them with `Object` (or the bound) in the compiled bytecode, which is why you can't do `list instanceof List<String>` at runtime.

Multithreading

Definition: The ability to execute multiple threads (lightweight subprocesses) concurrently within a single program to improve performance and responsiveness.

Creating Threads

```
// Method 1: extending Thread
class MyThread extends Thread {
    public void run() { System.out.println("Thread running"); }
}
new MyThread().start();

// Method 2: implementing Runnable (preferred – allows extending other cla
class MyTask implements Runnable {
    public void run() { System.out.println("Task running"); }
}
new Thread(new MyTask()).start();

// Method 3: Lambda (Java 8+)
Thread t = new Thread(() -> System.out.println("Lambda thread"));
```

```
t.start();
```

Thread Lifecycle

NEW → RUNNABLE → RUNNING → (BLOCKED / WAITING / TIMED_WAITING) → TERMINATED

ExecutorService (preferred over manual thread management)

```
ExecutorService executor = Executors.newFixedThreadPool(4);
executor.submit(() -> System.out.println("Task executed"));
executor.shutdown();
```

Best Practice: Prefer `ExecutorService`/thread pools over manually creating `Thread` objects for production code — pooling reduces the overhead of thread creation/destruction and provides lifecycle management.

Common Mistake: Calling `run()` directly instead of `start()` — this executes the code on the *current* thread synchronously rather than spawning a new thread.

Interview Tip: Know the difference between `Runnable` (no return value, `run()`) and `Callable` (returns a value via `Future`, can throw checked exceptions, `call()`).

Synchronization

Definition: A mechanism to control access to shared resources by multiple threads, preventing race conditions and ensuring data consistency.

```
class Counter {
    private int count = 0;
```

```
    public synchronized void increment() {   // synchronized method
        count++;
    }

    public void safeIncrement() {
        synchronized (this) {                // synchronized block
            count++;
        }
    }
}
```

Key concepts: - **Intrinsic lock (monitor)** – every object has an implicit lock; `synchronized` acquires it. - **`volatile`** – ensures visibility of a variable's latest value across threads (but does **not** guarantee atomicity of compound operations like count++). - **`java.util.concurrent.atomic`** – classes like `AtomicInteger` provide lock-free, thread-safe operations for simple counters. - **`ReentrantLock`** – an explicit, more flexible alternative to `synchronized` (supports tryLock, fairness policies, interruptible locking).

```
AtomicInteger atomicCount = new AtomicInteger(0);
atomicCount.incrementAndGet();   // thread-safe without explicit locking
```

Best Practice: Keep synchronized blocks as small as possible (synchronize only the critical section) to minimize contention and maximize throughput. Prefer high-level concurrency utilities (`java.util.concurrent`) over manual `synchronized`/`wait`/`notify` where possible.

Common Mistake: Synchronizing on a mutable or non-final field (the lock reference can change, breaking mutual exclusion), or forgetting that `count++` is not atomic even if `count` is `volatile` — it involves a read, increment, and write, requiring `synchronized` or `AtomicInteger`.

Interview Tip: Be ready to explain **deadlock** (two or more threads waiting on locks held by each other) and how to avoid it: acquire locks in a consis-

tent global order, use timeouts (`tryLock`), or reduce lock granularity.

File Handling

Definition: Java provides APIs (`java.io` and `java.nio.file`) to create, read, write, and manipulate files.

Legacy `java.io` example:

```
import java.io.*;

try (BufferedWriter writer = new BufferedWriter(new FileWriter("output.txt
    writer.write("Hello, File!");
} catch (IOException e) {
    e.printStackTrace();
}

try (BufferedReader reader = new BufferedReader(new FileReader("output.txt
    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

Modern `java.nio.file` (Java 7+, recommended):

```
import java.nio.file.*;
import java.util.List;

Path path = Paths.get("output.txt");
Files.write(path, "Hello NIO".getBytes());
```

```java
List<String> lines = Files.readAllLines(path);
boolean exists = Files.exists(path);
```

Best Practice: Always use try-with-resources for streams/readers/writers so they are closed automatically, even if an exception occurs. Prefer `java.nio.file` (`Files`, `Path`) over legacy `java.io.File` for new code — it offers a richer, more consistent API.

Common Mistake: Forgetting to close streams (resource leaks) when not using try-with-resources, and not handling `IOException` properly (e.g., swallowing it silently).

Interview Tip: Know the difference between byte streams (`InputStream`/`OutputStream`, for binary data) and character streams (`Reader`/`Writer`, for text data with proper encoding handling).

Streams API

Definition: Introduced in Java 8, the Streams API (`java.util.stream`) enables functional-style operations (map, filter, reduce) on sequences of elements from collections, arrays, or I/O sources, without modifying the source.

```java
import java.util.*;
import java.util.stream.*;

List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6, 7, 8);

List<Integer> evenSquares = numbers.stream()
        .filter(n -> n % 2 == 0)     // intermediate operation
        .map(n -> n * n)             // intermediate operation
        .collect(Collectors.toList()); // terminal operation
```

```
System.out.println(evenSquares);     // [4, 16, 36, 64]

int sum = numbers.stream().mapToInt(Integer::intValue).sum();
Optional<Integer> max = numbers.stream().max(Integer::compareTo);

Map<Boolean, List<Integer>> partitioned = numbers.stream()
        .collect(Collectors.partitioningBy(n -> n % 2 == 0));
```

Key concepts: - **Intermediate operations** (`map`, `filter`, `sorted`, `distinct`) – lazy, return a new stream. - **Terminal operations** (`collect`, `forEach`, `reduce`, `count`) – trigger actual execution and produce a result or side effect. - **Streams are single-use** – once a terminal operation runs, the stream is consumed and cannot be reused. - **Parallel streams** (`.parallelStream()`) – process elements concurrently using the common `ForkJoinPool`, useful for CPU-bound operations on large datasets.

Best Practice: Prefer streams for readable, declarative data transformations, but avoid overusing them for simple loops where a traditional `for` loop is clearer and equally performant. Avoid side effects (e.g., modifying external state) inside stream lambda expressions.

Common Mistake: Trying to reuse a stream after a terminal operation has been called, which throws `IllegalStateException`. Also, using `parallelStream()` on small collections, where thread coordination overhead outweighs any performance benefit.

Interview Tip: Be ready to explain **laziness**: intermediate operations don't execute until a terminal operation is invoked — this allows the JVM to optimize the entire pipeline (e.g., short-circuiting with `findFirst()`).

Lambda Expressions

Definition: A concise way to represent an anonymous function (a block of code with parameters) that can be passed around as a value — introduced in Java 8, primarily used to implement functional interfaces.

Syntax: `(parameters) -> expression` or `(parameters) -> { statements; }`

```
// Before Java 8 (anonymous inner class)
Runnable r1 = new Runnable() {
    public void run() { System.out.println("Running..."); }
};

// With lambda
Runnable r2 = () -> System.out.println("Running...");

Comparator<String> byLength = (a, b) -> a.length() - b.length();

List<String> names = new ArrayList<>(List.of("Charlie", "Alice", "Bob"));
names.sort((a, b) -> a.compareTo(b));

// Method reference (shorthand for a lambda that calls an existing method)
names.forEach(System.out::println);
```

Best Practice: Keep lambda bodies short and focused; extract complex logic into a named method and reference it (`ClassName::methodName`) for readability.

Common Mistake: Trying to modify a local variable from an enclosing scope inside a lambda — local variables captured by a lambda must be **effectively final** (never reassigned after initialization).

Interview Tip: Be ready to explain why lambdas require a **functional interface** as their target type — the lambda's parameter list and body

must match the single abstract method's signature.

Functional Interfaces

Definition: An interface with exactly **one abstract method** (may have multiple `default`/`static` methods), which can be implemented using a lambda expression or method reference. Marked (optionally but recommended) with `@FunctionalInterface`.

```
@FunctionalInterface
interface Calculator {
    int operate(int a, int b);
}

Calculator add = (a, b) -> a + b;
Calculator multiply = (a, b) -> a * b;
System.out.println(add.operate(3, 4));       // 7
System.out.println(multiply.operate(3, 4));  // 12
```

Common built-in functional interfaces (`java.util.function`):

Interface	Method	Purpose
`Function<T,R>`	`R apply(T t)`	transform input to output
`Predicate<T>`	`boolean test(T t)`	boolean-valued check
`Consumer<T>`	`void accept(T t)`	perform an action, no return
`Supplier<T>`	`T get()`	supply/produce a value, no input
`BiFunction<T,U,R>`	`R apply(T t, U u)`	two inputs, one output
`UnaryOperator<T>`	`T apply(T t)`	input and output of the same type

```java
Predicate<Integer> isEven = n -> n % 2 == 0;
Function<Integer, Integer> square = n -> n * n;
Supplier<String> greeting = () -> "Hello!";
Consumer<String> printer = System.out::println;
```

Best Practice: Reuse the standard `java.util.function` interfaces instead of defining custom ones whenever the shape (arity/return type) matches — this improves interoperability with the Streams API and other libraries.

Common Mistake: Defining an interface with more than one abstract method and expecting it to work as a lambda target — this is a compile-time error (`@FunctionalInterface` will flag it explicitly if used).

Interview Tip: Be ready to write a custom functional interface and immediately use it with a lambda — a frequent hands-on coding question.

Java 8 Features

Definition: Java 8 (2014) introduced major language and API enhancements that remain central to modern Java interviews.

Key features: 1. **Lambda Expressions** – concise anonymous functions (see dedicated section above). 2. **Functional Interfaces & `java.util.function`** – standard single-method interface library. 3. **Streams API** – functional-style data processing pipelines. 4. **Default and Static methods in interfaces** – allow interface evolution without breaking implementers. 5. **`Optional<T>`** – a container object that may or may not hold a non-null value, used to reduce `NullPointerException` risk. `java Optional<String> name = Optional.ofNullable(getName()); String result = name.orElse("Default"); name.ifPresent(System.out::println);` 6. **New Date/Time API (`java.time`)** – immutable, thread-safe replace-

ment for the flawed old `Date`/`Calendar` classes. `java LocalDate today = LocalDate.now(); LocalDate birthday = LocalDate.of(1995, 5, 20); Period age = Period.between(birthday, today);` 7. **Method References** – shorthand syntax for calling an existing method: `ClassName::staticMethod`, `instance::method`, `ClassName::new`. 8. **`Collectors` utility class** – rich terminal operations for streams (`toList()`, `groupingBy()`, `joining()`, `summingInt()`).

Best Practice: Use `Optional` as a **return type** to signal "may be absent" (never as a field type or method parameter — this is considered an anti-pattern). Use `java.time` classes exclusively for new date/time logic.

Common Mistake: Calling `.get()` on an `Optional` without checking `.isPresent()` first — this defeats the purpose and can still throw `NoSuchElementException`. Prefer `.orElse()`, `.orElseGet()`, or `.ifPresent()`.

Interview Tip: Be ready to discuss why Java 8 was considered a "functional programming" milestone for Java, and how `Stream` + lambdas + method references work together in a typical data-processing pipeline.

Memory Management

Definition: Java manages memory automatically through the JVM, dividing runtime memory into distinct areas.

Runtime memory areas: - **Heap** – stores all objects and instance variables; shared across threads; divided into **Young Generation** (Eden + Survivor spaces) and **Old Generation (Tenured)**. - **Stack** – stores method call frames, local variables, and partial results; each thread has its own stack; automatically reclaimed when a method returns. - **Method Area (Metaspace since Java 8)** – stores class metadata, static variables, constant pool. - **PC (Program Counter) Register** – holds the address of the currently executing instruction per thread. - **Native Method Stack** – sup-

ports native (non-Java) method calls.

```
void demo() {
    int localVar = 10;           // stored on the Stack
    Object obj = new Object();   // reference on Stack, actual object on H
}
```

Best Practice: Avoid unnecessary object creation inside loops or hot code paths; reuse objects (e.g., via object pools or `StringBuilder`) when appropriate to reduce GC pressure.

Common Mistake: Confusing stack and heap storage — primitives and references live on the stack, but the actual objects they point to always live on the heap. Also, causing memory leaks by holding references in long-lived collections (e.g., static `Maps`) that are never cleared.

Interview Tip: Be ready to explain `StackOverflowError` (excessive/infinite recursion exhausting stack space) vs `OutOfMemoryError` (heap exhausted, too many live objects retained).

Garbage Collection

Definition: The automatic process by which the JVM identifies and reclaims memory occupied by objects that are no longer reachable from any live thread/root reference.

How it works (generational GC): 1. New objects are allocated in the **Young Generation (Eden space)**. 2. Objects surviving multiple minor GC cycles are promoted to the **Old Generation**. 3. **Minor GC** – cleans the Young Generation (fast, frequent). 4. **Major/Full GC** – cleans the Old Generation (slower, less frequent, can cause noticeable pauses).

Common GC algorithms: Serial GC, Parallel GC, CMS (deprecated), **G1 GC** (default since Java 9, balances throughput and pause time), **ZGC** and

Shenandoah (low-latency collectors for very large heaps).

Reachability & eligibility for GC: An object becomes eligible for garbage collection when it is no longer reachable through any chain of references from GC roots (active threads, static references, local variables in scope).

```
Object obj = new Object();
obj = null;   // original object now eligible for GC (if no other referenc
```

`finalize()` (deprecated, avoid using): A method historically called before GC reclaims an object — unreliable timing, deprecated since Java 9. Use `try-with-resources`/`AutoCloseable` instead for deterministic resource cleanup.

Best Practice: Don't manually call `System.gc()` in production code — it's only a *hint* to the JVM and rarely improves performance; let the JVM's adaptive GC algorithms manage memory. Nullify references to large objects when they are no longer needed in long-lived scopes to help the GC reclaim memory sooner.

Common Mistake: Believing `System.gc()` guarantees immediate garbage collection — it does not; it merely suggests the JVM run a collection cycle. Also, assuming Java is immune to memory leaks — leaks can still occur via lingering references (e.g., static collections, unclosed resources, listener registrations never removed).

Interview Tip: Be ready to name and briefly describe at least two GC algorithms (e.g., G1 vs. CMS) and to explain the general concept of "stop-the-world" pauses during garbage collection.

Common Interview Mistakes

A consolidated list of frequent pitfalls candidates make in Java interviews — useful as a quick-reference checklist.

1. **Confusing == and `.equals()`** for object/String comparison — == compares references, `.equals()` compares logical content (when properly overridden).
2. **Mixing up overloading and overriding** — overloading is compile-time (same class, different parameters); overriding is runtime (subclass, same signature).
3. **Thinking JDK and JRE are interchangeable** — JDK includes development tools (compiler); JRE only runs compiled code.
4. **Forgetting `String` immutability** — assuming string methods mutate the original object in place.
5. **Not knowing why `String` is a good `HashMap` key** — immutability guarantees a stable hash code.
6. **Misunderstanding `final`, `finally`, and `finalize()`** — `final` (keyword for constants/no-override/no-extend), `finally` (block that always executes), `finalize()` (deprecated GC hook).
7. **Assuming `HashMap`/`HashSet` maintain insertion order** — they do not; use `LinkedHashMap`/`LinkedHashSet` instead if needed.
8. **Not knowing that arrays and collections handle generics differently** due to type erasure and the inability to create generic arrays directly.
9. **Believing `synchronized` alone prevents all concurrency issues** — visibility, atomicity, and ordering must all be considered (e.g., `volatile` vs `synchronized` vs `Atomic*`).
10. **Ignoring exception hierarchy** — not distinguishing checked vs. unchecked exceptions, or catching overly broad `Exception`/`Throwable`.
11. **Struggling to explain the diamond problem** and how default methods in interfaces require explicit resolution when conflicts arise.
12. **Forgetting `this()`/`super()` must be the first line** in a constructor, and that they cannot both be used in the same constructor.
13. **Not knowing `static` methods cannot be overridden**, only hidden (resolved at compile time based on reference type).

14. **Underestimating autoboxing costs** — e.g., unboxing a `null Integer` throws `NullPointerException`, and Integer caching (-128 to 127) can cause surprising == results.
15. **Confusing `Comparable` (natural ordering, `compareTo`, implemented by the class itself) with `Comparator`** (external ordering strategy, `compare`, can define multiple orderings).

Interview Tip: When unsure of an answer, verbalize your reasoning process rather than guessing silently — interviewers value structured thinking even when the final answer isn't perfect.

Best Practices

A consolidated checklist of Java best practices commonly expected of professional developers.

- **Follow naming conventions:** `PascalCase` for classes/interfaces, `camelCase` for methods/variables, `UPPER_SNAKE_CASE` for constants.
- **Favor immutability** where practical — immutable objects are inherently thread-safe and easier to reason about.
- **Program to interfaces**, not implementations (`List<String> list = new ArrayList<>();`).
- **Use `try-with-resources`** for anything implementing `AutoCloseable` to guarantee resource cleanup.
- **Validate constructor/setter inputs** to protect object invariants (encapsulation in practice).
- **Override `equals()` and `hashCode()` together**, and keep them consistent with each other.
- **Avoid raw types** — always use parameterized generics (`List<String>`, not raw `List`).
- **Handle exceptions meaningfully** — catch specific exception types, log or rethrow with context, avoid empty `catch` blocks.

- **Minimize the scope/visibility** of fields and methods (`private` by default, widen only when necessary).
- **Use `StringBuilder`** for string concatenation in loops instead of the + operator.
- **Leverage the Streams API** for readable, declarative collection processing, but don't force it where a simple loop is clearer.
- **Write unit tests** (e.g., JUnit) for business logic, and design classes with testability in mind (dependency injection over hard-coded dependencies).
- **Avoid premature optimization** — write clear, correct code first, then profile and optimize actual bottlenecks.
- **Document public APIs** with Javadoc comments (`/** ... */`) explaining purpose, parameters, return values, and exceptions.
- **Keep methods short and focused** (single responsibility) to improve readability, testability, and maintainability.

Interview Tip: When discussing best practices, tie each one to a concrete *reason* (thread-safety, readability, performance, maintainability) rather than reciting rules — this demonstrates genuine understanding rather than memorization.

End of Knowledge Base.